



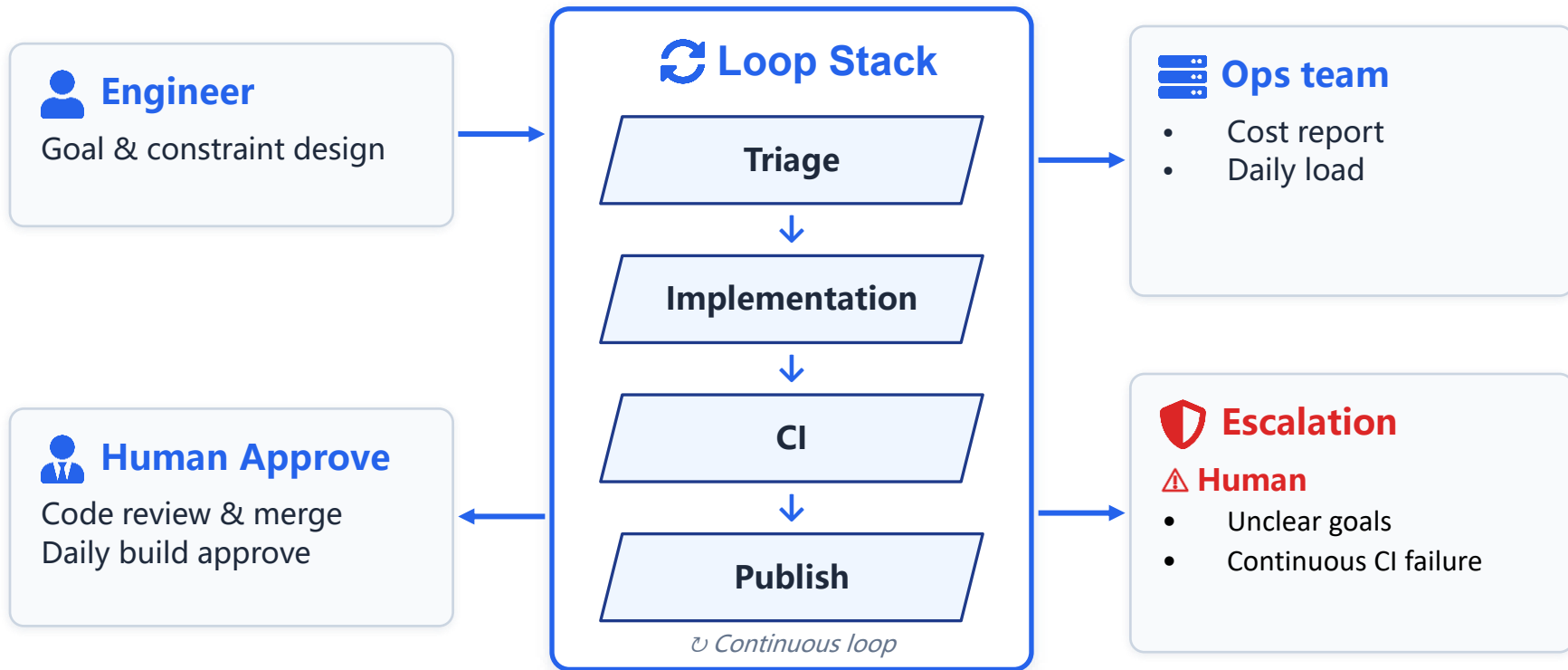
Loop Engineering

R&D Team · Product Introduction

Operating model: the development team manages humans, agents, loops, and feedback systems

R&D Team: Loop Stack Architecture

Operating model: the development team manages humans, agents, loops, and feedback systems



Key Human Activities

Humans set direction and standards, guarding four key gates in the loop



GIT Issues

Enhancement & Bug

Goal setting

By engineers

Acceptance rules

By experts

Engineer + Expert



Token Budget

Budget & Monitoring

Budget setting

Cost ceiling

Usage monitoring

By engineers

Owner: Engineer



Merge Review

Pull Requests

Code quality gate

Human review

Merge decision

By experts

Owner: Expert



Daily Build

Build & Release

Build result review

By engineers

Release decision

By engineers

Owner: Engineer

Agents run the loop automatically between the four gates; humans step in only at key decision points to control direction, cost, and quality.

Five Engineering Roles

Where it fits: Loop Engineering adds value across roles throughout the product lifecycle



Prototyper

Generates brand-new ideas and produces a high volume of concepts quickly — most of which never ship.



Builder

Rapidly turns a prototype or idea into a production-grade product or infrastructure.



Sweeper

Polishes UI, simplifies code and systems, retires unnecessary features, and optimizes performance.



Grower

Keeps iterating after the product is built to improve product-market fit (PMF).



Maintainer

Keeps mature systems running long-term — safe, reliable, fast, and efficient as they scale.

Loop Mechanics ①: Skill Orchestration

8 AI skills + 3 human gates, coordinated via GIT issue status flow and STATUS.md signals

Triage

1

Evaluate Open tickets:
prioritize, estimate effort,
generate the
implementation list

Implementation

2

Implement tickets from the
list one by one; an
independent agent verifies;
output PRs

Retrospective

3

Review rejected items,
update / create skills, then
re-enter the loop

CI

4

Independent CI agent
builds, tests, and validates
on the main branch

CI-Sweeper

5

Auto-fix failed CI and rerun
the loop until it passes

CI-Retrospective

6

Distill CI failure lessons into
skills, then re-enter the fix
loop

Publish

7

Archive approved CI
artifacts and close related
tickets

Cost-report

8

Aggregate token usage by
ticket type and processing
stage

Three human gates

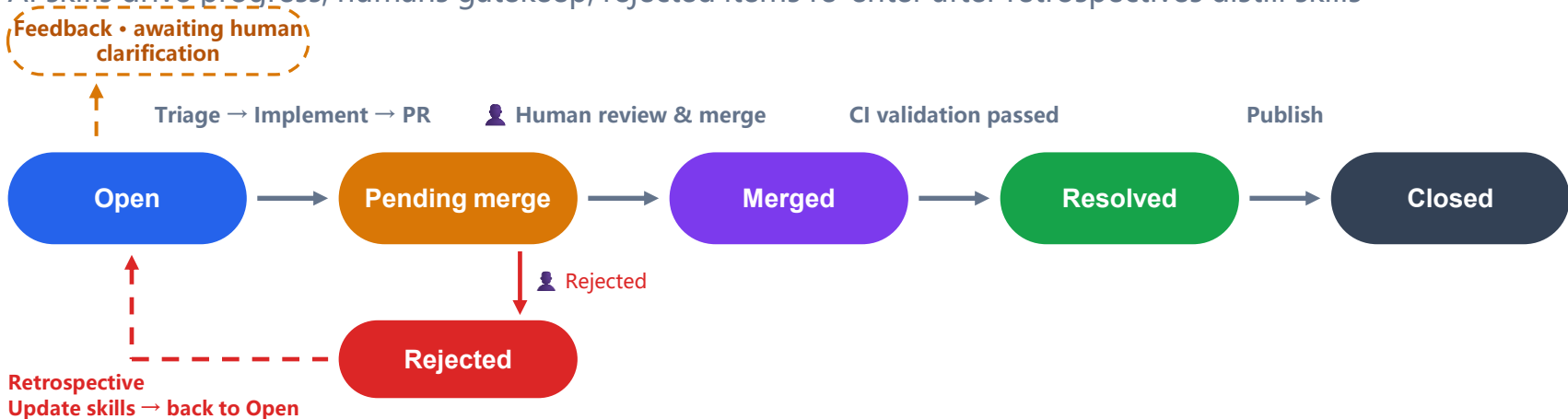
① PR code review & merge ② Escalation incident fix ③ CI result approval

STATUS.md coordination signals

STOP = pause loops CI = mutual-exclusion avoidance

Loop Mechanics ②: Feature / Bug Loop

AI skills drive progress, humans gatekeep; rejected items re-enter after retrospectives distill skills



Triage essentials

- Prioritize Open tickets that lack a priority
- Estimate effort (later converted to estimated tokens)
- Move ambiguous items to Feedback for human clarification
- Record Analysis Token Usage; produce the implementation list

Implementation essentials

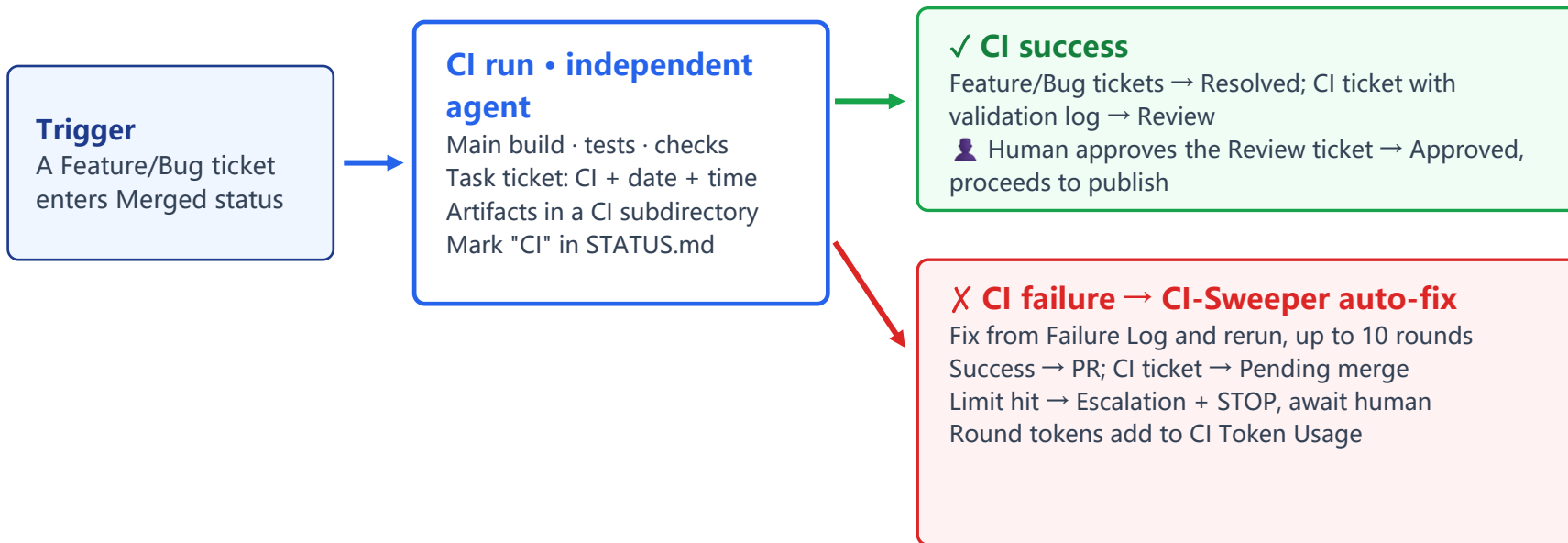
- Per ticket: worktree → implement → verify → PR
- Verification must be done by an independent agent
- Write the PR link back to the ticket; status → Pending merge
- Check STATUS.md before starting; skip the round on STOP / CI

Retrospective essentials

- Read all Feature/Bug tickets in Rejected status
- Update / create skills based on rejection reasons
- Write skill-change notes and retrospective tokens back to the ticket
- Reset the ticket to Open for the next implementation round

Loop Mechanics ③: CI Validation Loop (Task)

Each CI run is tracked by a Task ticket: success goes to approval & release; failures are auto-fixed by CI-Sweeper in a closed loop



Closing the loop: 👤 a human fixes the Escalation incident and records the solution → CI-Retrospective distills the lesson into a skill → the ticket returns to Failed for re-validation; regular CI resumes once the human removes STOP.

Loop Mechanics ④: Publish, Sync & Cost

Archive & release after approval; STATUS.md coordinates parallel loops; cost reporting runs throughout



Publish

- Triggered by CI tickets in Approved status
- Also processes earlier CI tickets still in Review
- Copies the CI artifacts to the publish/ subdirectory as an approval record
- Related Feature/Bug and Task tickets all move to Closed



STATUS.md coordination signals

- "CI": CI in progress — Implementation skips this round
- "STOP": escalated incident — CI and CI-Sweeper pause
- Loops resume automatically once humans resolve and remove STOP
- Every skill reads STATUS.md before starting work



Cost-report

- Aggregates token usage across all tickets
- Dimensions: ticket type × processing stage
- Stage fields: Analysis / Implementation / CI / Retrospective Token Usage
- Supports continuous monitoring for the Token Budget gate

Loop Mechanics ⑤: GIT Ticket Data Model

Ticket types, the full status set, and field definitions on GitHub Projects — the unified data foundation of the loop

Ticket types (3)

Feature

Bug

Task

The Task type is dedicated to CI activity tracking

Status set (12)

Create

Open

Pending merge

Merged

Resolved

Closed

Feedback

Rejected

Review

Approved

Failed

Escalation

Built-in fields (GitHub native)

- Title
- Issue body
- Type
- Issue author
- Assignees
- Updated
- Activity log + timeline
- Priority: Urgent / High / Medium / Low
- Effort: Low / Medium / High

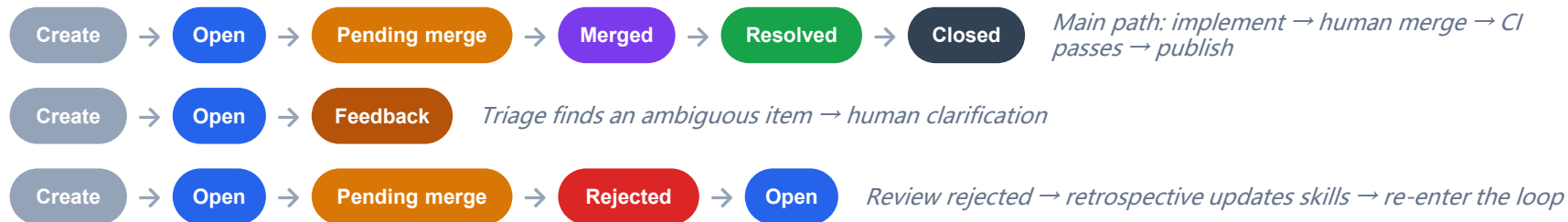
Custom fields (loop-specific)

- Status (loop status)
- Estimate Token (expected usage)
- Budget Token (budget cap)
- Detailed Description
- Failure Log
- Analysis Token Usage
- Implementation Token Usage
- Retrospective Token Usage
- CI Token Usage
- (four-stage usage feeds the cost report)

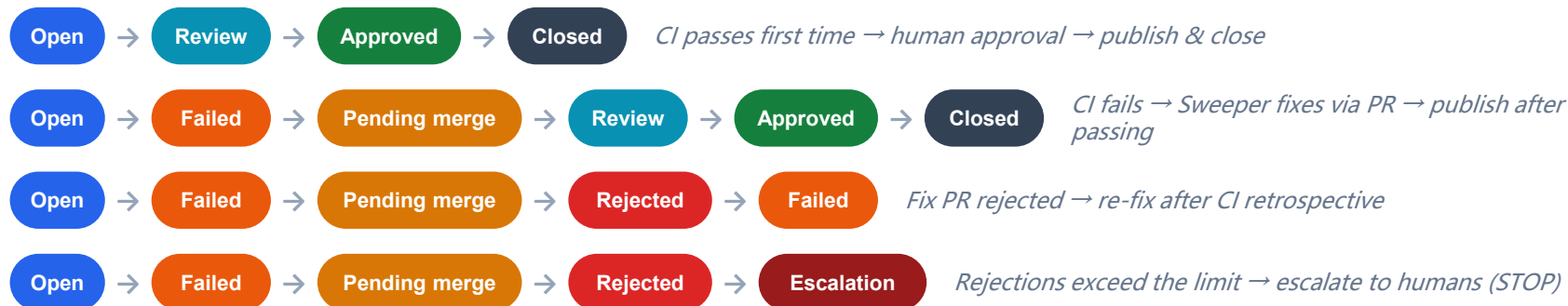
Loop Mechanics ⑥: Status Flow Paths

Feature / Bug Ticket 3 paths · Task (CI) Ticket 4 paths

Feature / Bug Ticket



Task Ticket (CI activity)



Loop Mechanics ⑦: Scheduling Cadence

Set frequency by each skill' s trigger source: AI-driven skills run often, human-dependent ones daily, reports weekly

<code>/loop 4h /triage</code>	6× per day	Keeps the implementation queue fresh; runs more often than Implementation, with staggered starts
<code>/loop 6h /implementation</code>	4× per day	Core output loop; checks STATUS.md before starting to steer clear of CI
<code>/loop 24h /retrospective</code>	1× per day	Rejections come from human review; a daily consolidated retrospective suffices
<code>/loop 12h /ci</code>	2× per day	Main-branch validation; matches the human merge cadence (weekdays)
<code>/loop 6h /ci-sweeper</code>	4× per day	Starts fixes within 6 hours of a CI failure to unblock quickly
<code>/loop 24h /ci-retrospective</code>	1× per day	Escalations depend on humans; distill lessons once a day
<code>/loop 24h /publish</code>	1× per day	Fixed daily release window; archives approved CI artifacts
<code>/loop 7d /cost-report</code>	1× per week	Weekly cost report supporting the Token Budget gate review

Blue = feature loop Purple = CI loop Green = delivery & cost / Idle runs cost almost nothing: loops exit immediately when no tickets are pending

Token Optimization: Complexity Routing

Prompts are scored and routed to four complexity tiers, each served by an independent model configuration



Simple

Short factual queries,
formatting,
classification tasks



Medium

Summarization, text
drafting, routine
Q&A



Complex

Multi-step analysis,
long context, code
generation



Reasoning

Deep chain-of-
thought, planning,
math & logic



Model per tier

Any model can be mapped to any
complexity tier



Automatic failover

Falls back to a backup model
when the primary is unavailable



Customizable criteria

Complexity thresholds and
classification rules configurable
per customer

Token Optimization: Per-Agent Models

Agents with different duties use models at different cost–performance points, reserving high-capability models for critical steps

opencode.json · agent configuration

```
"agent": {
  "chat": {
    "model": "opencode/deepseek-v4-flash-free"
  },
  "plan": {
    "model": "sicore-deepseek/deepseek-v4-pro"
  },
  "build": {
    "model": "opencode/deepseek-v4-flash-free"
  },
  "review": {
    "model": "sicore-ali/qwen3.7-max"
  },
  "commit": {
    "model": "sicore-zhipu/glm-5.1"
  }
},
```

chat [deepseek-v4-flash-free](#)

Lightweight free model handles high-frequency interaction

plan [deepseek-v4-pro](#)

High-capability model handles critical task decomposition

build [deepseek-v4-flash-free](#)

Free model carries the high-volume build loop

review [qwen3.7-max](#)

Flagship model strictly gatekeeps code quality

commit [glm-5.1](#)

Mid-tier model generates well-formed commit messages

Model Inference Options

Four access modes to choose from — from bring-your-own-key to on-prem frontier models



BYO API Key

Bring Your Own Key

Your own model API keys — full control of choice and cost

User keys



Cloud API

Managed Cloud API

Cloud-model API keys from us — zero ops

We provide

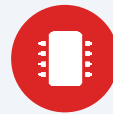


HERO TX

On-Prem · Mid-Size

Mid-size models on-prem; data never leaves your network

HERO product



HERO RX

On-Prem · Frontier

Frontier models on-prem; fully private

HERO product

Increasing data privacy and autonomous control



Humans set direction & standards

Agents execute the loop

Four human gates × five engineering roles × flexible model inference — shifting dev teams from writing code to managing loops